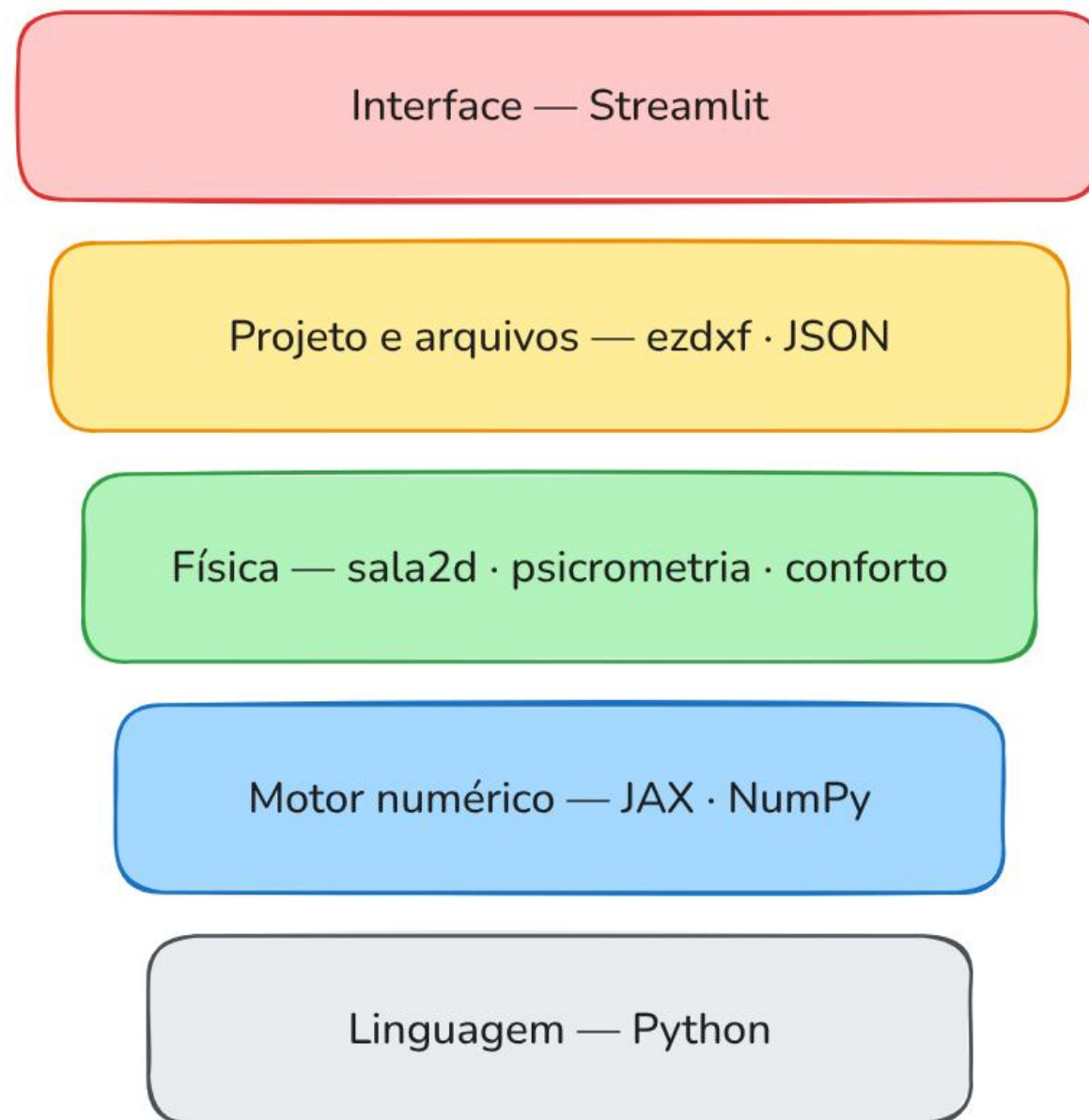
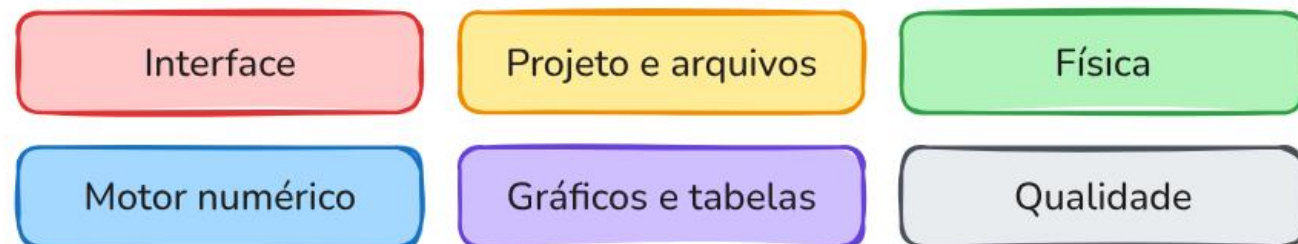


As ferramentas por trás do Clima2D

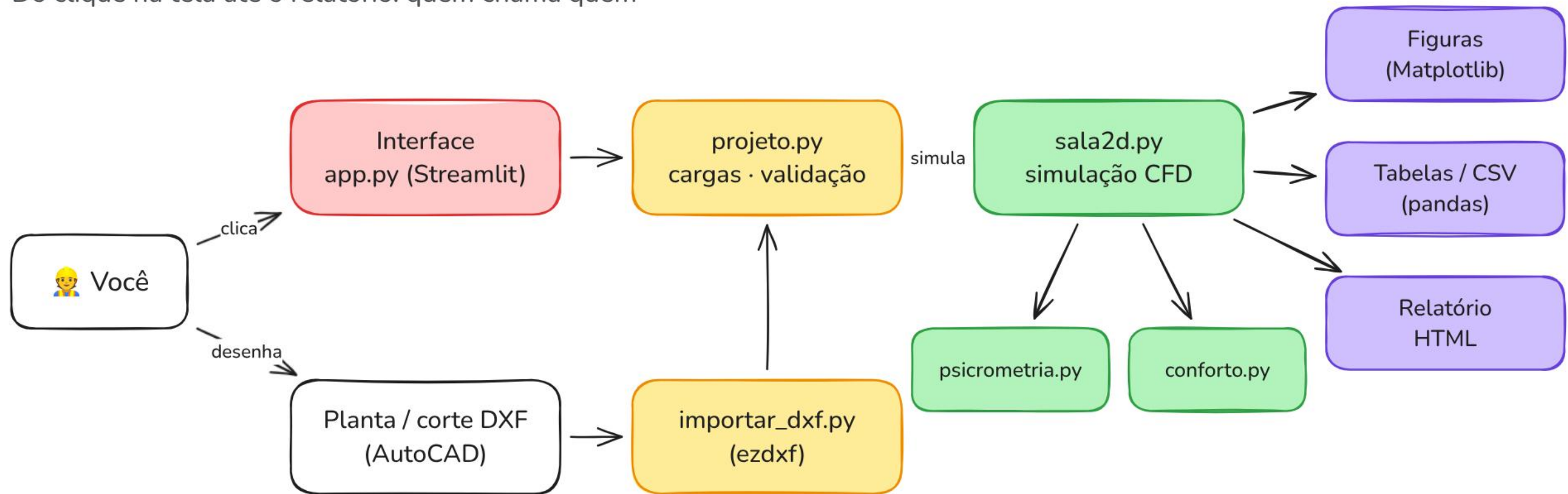
Um mapa para aprender, ferramenta por ferramenta,
o que faz cada peça do estudo de climatização.

Cada cor é uma camada:



Como as peças se conectam

Do clique na tela até o relatório: quem chama quem



Motor numérico: JAX + NumPy (tudo acima roda sobre ele)

Python: a linguagem que une tudo

E três peças que fazem o programa rodar igual em qualquer computador



Python

A linguagem.
Todo o Clima2D é escrito nela.



pip

A "loja" de bibliotecas.
Baixa NumPy, JAX, Streamlit...



venv (.venv)

Uma caixa isolada com as
versões certas, só deste projeto.



requirements.txt

A lista de compras:
quais bibliotecas instalar.

O que o iniciar.bat faz por você:

cria a .venv
(1ª vez)



pip install
-r requirements.txt



streamlit run
app\app.py



navegador abre
localhost:8501

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```



Analogia: a .venv é como uma maleta de ferramentas só deste serviço. Atualizar algo em outro projeto não estraga este.

NumPy: contas com a tabela inteira de uma vez

A base de quase toda a ciência em Python

A sala vira uma grade de números:

25.8	25.9	26.1	26.3
24.6	24.4	24.5	24.9
23.1	22.6	22.8	23.4
21.9	21.5	21.7	22.2

$T[i, j]$ = temperatura da célula (i, j)
na simulação real: $50 \times 28 = 1.400$ células

```
import numpy as np
T = np.full((50, 28), 24.0) # sala inteira a 24 °C
T[:, 0] -= 2                # piso 2 °C mais frio
media = T.mean()            # uma linha, sem laço
```

⚡ Por que importa: o Excel calcula célula por célula; o NumPy opera a grade inteira de uma vez, em código compilado — dezenas a centenas de vezes mais rápido que um laço em Python.

📍 Onde usamos no Clima2D

- guardar os campos de temperatura, umidade e velocidade
- calcular médias na zona ocupada
- exportar o CSV e alimentar as figuras

JAX: o NumPy com superpoderes

Biblioteca do Google para computação científica e aprendizado de máquina

jnp

Mesma sintaxe do NumPy:
jnp.mean, jnp.exp...

📌 todo o código físico

jax.jit

Compila a função: vira código
de máquina otimizado

📌 a simulação compila uma vez
e roda milhares de passos

jax.grad

Derivada EXATA de qualquer
função escrita em JAX

📌 sensibilidades (ex. 02)
otimização (ex. 04)

jax.lax.scan

Laço no tempo que pode ser
compilado e derivado

📌 os ~40 mil passos
de tempo da CFD

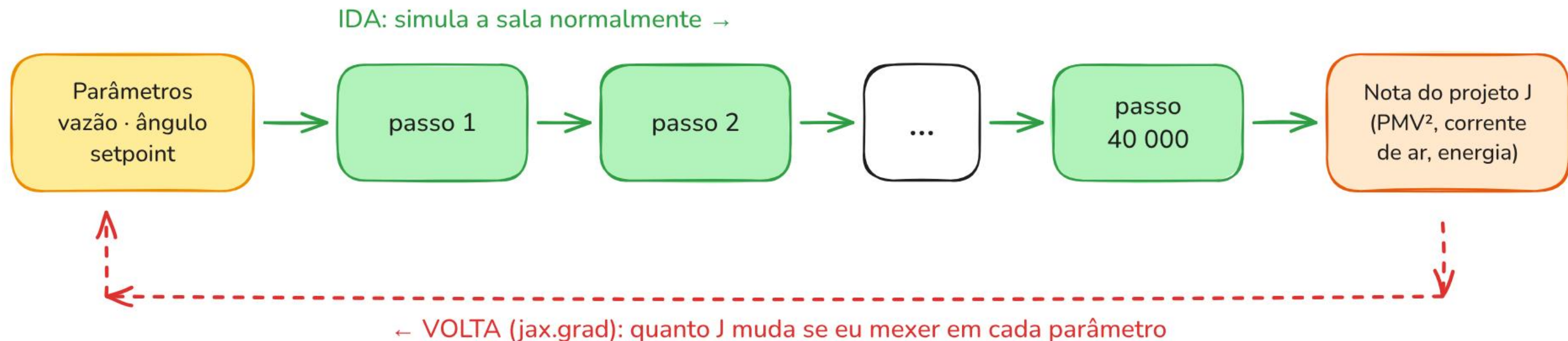
```
import jax
f = lambda t: psi.umidade_relativa(t, 0.010)
jax.grad(f)(24.0) # → -0.032
# a UR cai 3,2 % por °C de aquecimento
```

⚠️ Duas regras que mais dão erro

1. Arrays não mudam no lugar:
x = x.at[i].set(v) (e não x[i] = v)
2. Dentro do jit, sem "if" em valores:
use jnp.where(condição, a, b)

Como o jax.grad atravessa a simulação inteira

A ideia que permitiu otimizar vazão, ângulo e setpoint automaticamente



Custo

≈ 3 simulações, seja para
3 ou para 300 parâmetros



Memória

jax.checkpoint guarda só alguns
estados e recalcula o resto



Diferenças finitas

2 simulações POR parâmetro
(300 parâmetros = 600 simulações)

AeroJAX: de onde veio a ideia

Simulador 2D de aerodinâmica em JAX (Arno Meijer, licença LGPL v3)

✓ O que aproveitamos (a arquitetura)

- passo de tempo como função pura em JAX
- estado da simulação como "PyTree"
- laço no tempo com `jax.lax.scan`
- simulação inteira diferenciável (`jax.grad`)
- método da projeção de pressão
- obstáculos por penalização de Brinkman

🔧 O que precisou ser novo (climatização)

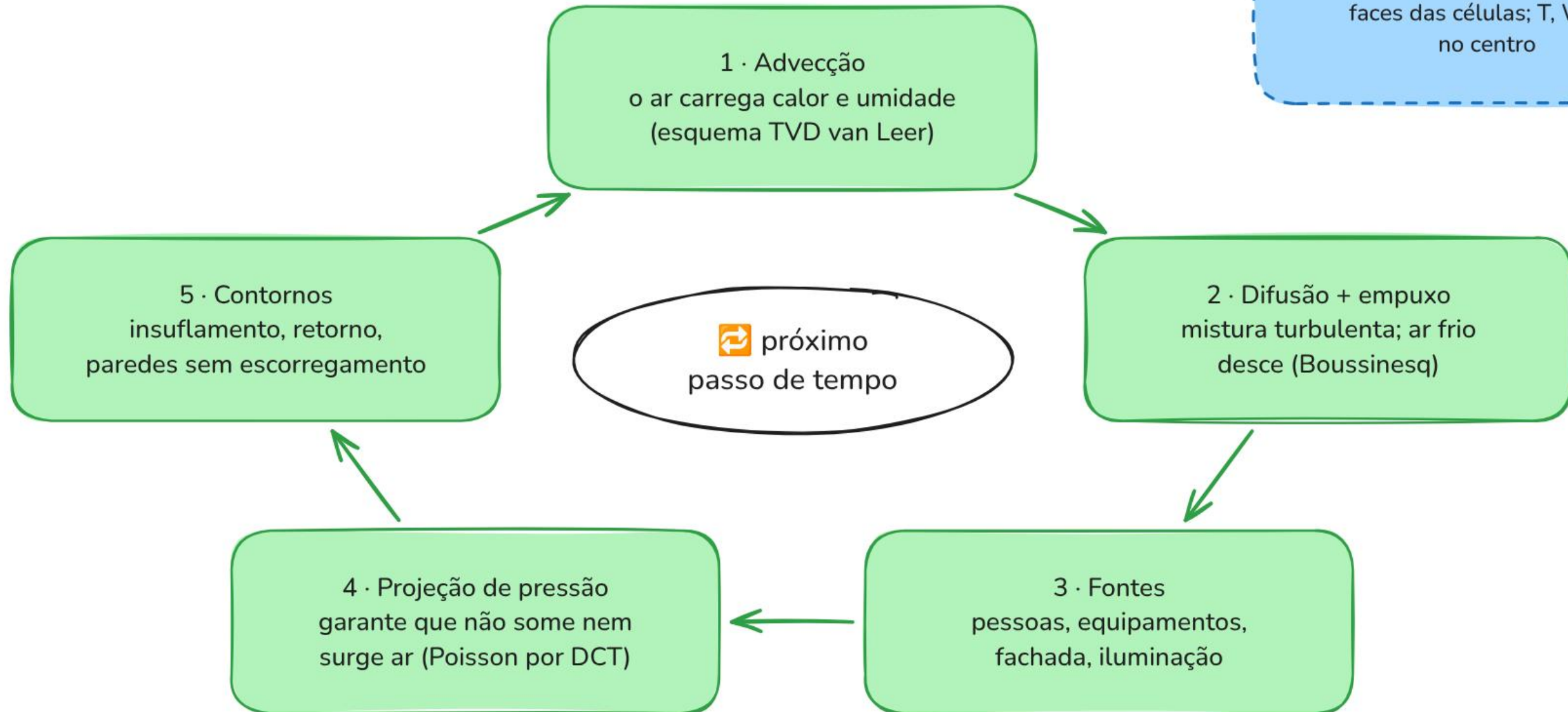
- sala fechada, aberturas em qualquer parede
- empuxo térmico: ar frio desce, quente sobe
- transporte de umidade
- cargas: pessoas, equipamentos, fachada
- conforto: PMV, PPD, corrente de ar
- interface, DXF e relatório

i O AeroJAX foi feito para escoamento externo (perfis NACA, cilindros). Ele foi estudado e rodado sem alterações — nada foi enviado ao repositório dele. O Clima2D reaproveita o jeito de construir, não o código.

Por dentro de um passo de tempo (sala2d.py)

Repetido ~40 mil vezes, cada um avançando 0,03–0,04 s

Malha MAC: velocidades nas faces das células; T, W e p no centro



Física do ar úmido e do conforto

Módulos escritos para o Clima2D, todos diferenciáveis com JAX

💧 psicrometria.py (ASHRAE Fundamentals)

- pressão de saturação (Hyland-Wexler)
- umidade absoluta ↔ relativa
- ponto de orvalho, bulbo úmido, entalpia
- mistura de ar externo + retorno
- serpentina: ADP e fator de bypass
- estado de insuflamento pelas cargas

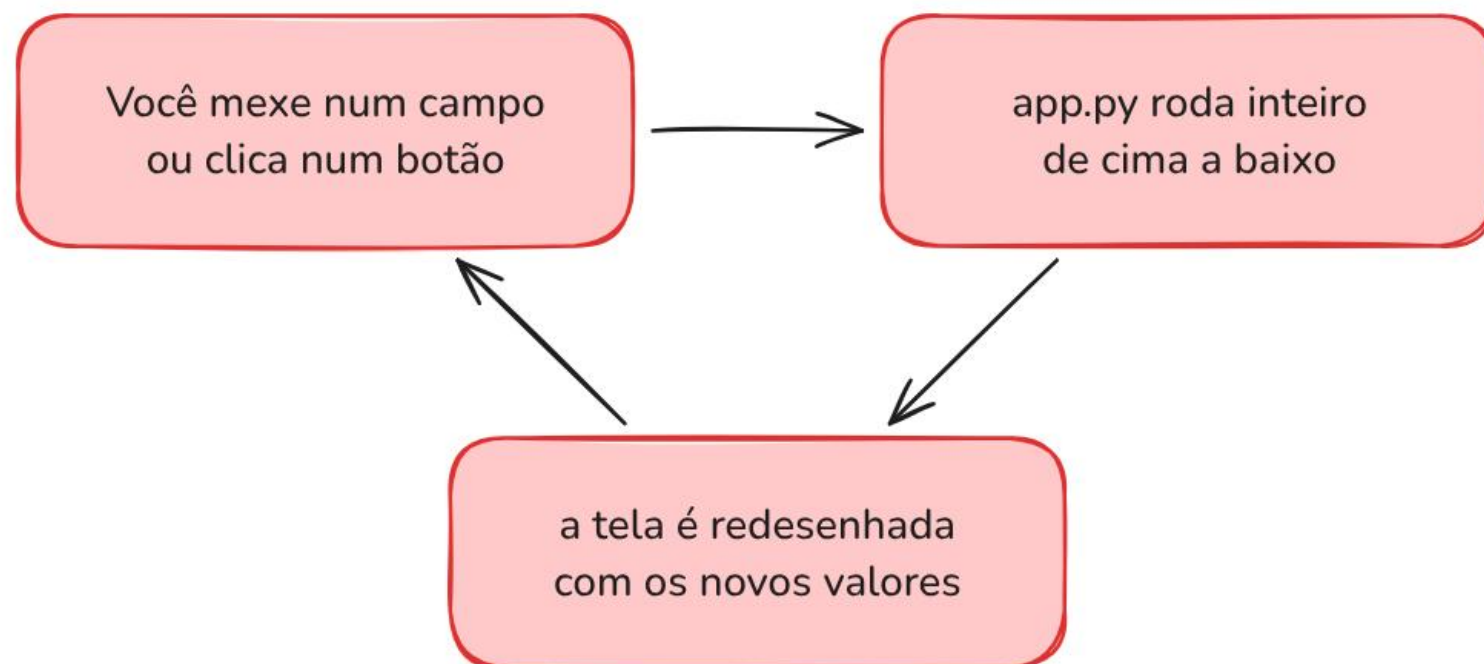
🧑 conforto.py (ISO 7730 / NBR 16401-2)

- PMV: voto médio estimado
(-3 muito frio · 0 neutro · +3 muito quente)
- PPD: % de pessoas insatisfeitas
- DR: risco de corrente de ar
("vento gelado" na pele)

✓ Validados contra: tabelas da ASHRAE · tabela D.1 da ISO 7730 · biblioteca de referência
pythermalcomfort · balanço de energia e umidade da própria simulação

Streamlit: interface web escrita só em Python

Sem HTML nem JavaScript: cada linha de Python vira um elemento na tela



A ideia central: o script inteiro roda a cada interação.

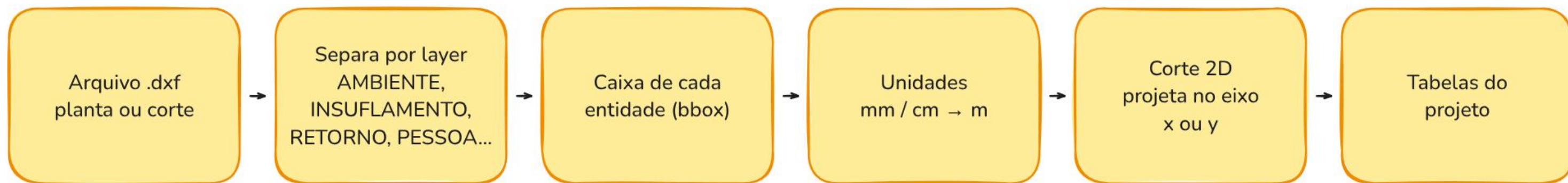
```
vazao = st.number_input("Vazão [m³/h]", value=600)
if st.button("▶ Simular"):
    r = simular_projeto(P)
    st.metric("PMV", r["PMV"])
```

🧠 `st.session_state`
memória que sobrevive às
reexecuções: o projeto fica aqui

🔒 Roda no SEU computador
localhost:8501 — nada vai
para a internet

ezdxf: lendo o desenho do AutoCAD

DXF é um formato aberto de CAD; o ezdxf lê e escreve sem precisar do AutoCAD



Por que layers?

O programa não "entende" o desenho como uma pessoa. A convenção de layers diz o que cada linha é: o contorno do ambiente, um difusor, uma pessoa, uma mesa.

Onde usamos no Clima2D

- importar_dxf.py: importar_corte e importar_planta
- a planta não tem alturas: pé-direito e alturas dos difusores vêm da interface
- gerar_exemplos cria os DXF de exemplo

Matplotlib e pandas: números viram figura e tabela

As duas bibliotecas mais usadas para mostrar resultados em Python

Matplotlib — figuras

- imshow: mapa de cores (temperatura, PMV...)
- streamplot: linhas do escoamento do ar
- contour: contorno da zona ocupada
- savefig: PNG para o relatório e o manual

```
fig, ax = plt.subplots()
ax.imshow(T.T, origin="lower", cmap="coolwarm")
ax.streamplot(x, y, u, v)
fig.savefig("sala.png")
```

pandas — tabelas

- DataFrame = uma planilha dentro do Python
- tabelas editáveis da interface (aberturas, fontes, obstáculos)
- exporta CSV com ";" e "," decimal, pronto para o Excel em português

```
df = pd.DataFrame({"x": x, "T": T})
df.to_csv("campos.csv", sep=";",
          decimal=",")
```

pytest: como saber que o cálculo está certo

Teste = uma afirmação verificável sobre o código

Integração: importar DXF, projeto JSON, validação

Numérica: divergência nula · gradiente = diferenças finitas

Conservação: energia e umidade fecham em regime ($\pm 2\%$)

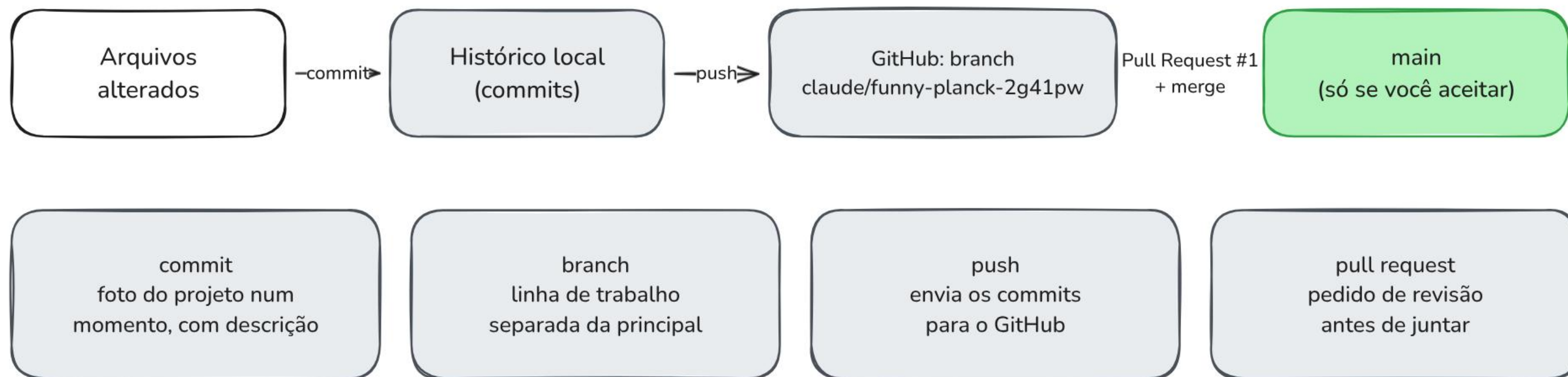
Propriedades: tabelas ASHRAE e ISO 7730

```
python -m pytest testes -q
..... 23 passed
```

⚠ Se você mudar o código e algum teste falhar,
o resultado das simulações não é mais confiável.

Git e GitHub: histórico e revisão

Cada mudança fica registrada e só entra no principal com a sua aprovação



🔒 AeroJAX original: apenas clonado para leitura. Nenhum commit foi enviado a ele.
Tudo foi para o SEU repositório allan01fr/allan01fr, num branch separado.

Ferramentas usadas para construir e conferir

Não fazem parte do Clima2D, mas ajudaram a garantir que ele funciona



Playwright

Navegador automático.
Abriu a interface, clicou nos botões, rodou a simulação e tirou as capturas do manual.



pythermalcomfort

Biblioteca de referência em conforto térmico. Usada só para conferir o nosso PMV.



Excalidraw + esbuild

Esta apresentação: os slides são gerados por um script e convertidos pelo próprio Excalidraw.



Claude Code

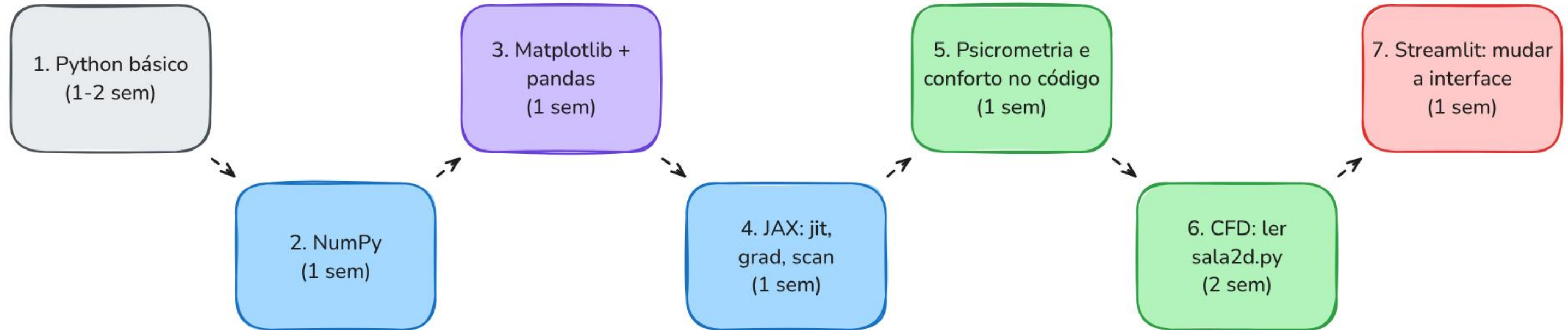
Ambiente onde o código foi escrito, testado e enviado para o GitHub.



Regra usada em todo o trabalho: nenhum número foi aceito sem conferência independente — tabela de norma, outra biblioteca, lei de conservação ou uma simulação mais longa. Foi assim que dois erros foram achados e corrigidos (otimização curta demais e NaN no gradiente).

Por onde começar a estudar

Uma trilha de ~8 semanas, 1 a 2 horas por dia



Material

- Python: docs.python.org/pt-br/3/tutorial
- NumPy: numpy.org/doc
- JAX: jax.readthedocs.io ("JAX 101")
- Streamlit: docs.streamlit.io
- ezdx: ezdx.mozman.at
- Matplotlib: matplotlib.org
- CFD: Ferziger & Perić, Computational Methods for Fluid Dynamics
- No repositório: README.md, módulos 0 a 5